Week 3: Authentication and Session Management

Learning Objectives

By the end of this lecture, you should be able to:

- *Distinguish authentication from authorization, and connect that distinction to the 401/403 codes from Week 2.*
- *Describe the three authentication factors and explain why combining them (MFA) is stronger than any one alone.*
- *Explain how passwords must be stored (salted, slow hashing) and why plaintext or fast hashes are unacceptable.*
- *Describe how session management works and why the session ID must be regenerated at login.*
- *Identify common authentication attacks: brute force, credential stuffing, session hijacking, session fixation, and their mitigations.*
- *Describe, at a high level, how token-based authentication (JWT) and OAuth/SSO differ from classic session cookies.*

Authentication vs. authorization

These two words get used almost interchangeably in casual conversation, but they answer different questions. Authentication (AuthN) answers "who are you?" logging in with a username and password proves your identity. Authorization (AuthZ) answers "what are you allowed to do?" once your identity is known, the system decides which resources and actions are permitted for that identity. This is exactly the distinction behind Week 2's 401 vs. 403: a 401 means authentication failed (identity unknown), a 403 means authentication succeeded but authorization failed (identity known, action denied).

Authentication factors

An authentication factor is a category of evidence used to prove identity. Relying on just one factor almost always "***something you know***", is the weakest setup, which is why Multi-Factor Authentication (MFA) combines factors from at least two different categories.

MFA is effective because it forces an attacker to defeat two unrelated categories at once. A leaked password alone is no longer enough if the account also requires a code from your phone.

Factor	Examples	Weakness if used alone
Something you know	Password, PIN, security question	Can be guessed, phished, or reused from another breach.
Something you have	Phone (OTP app/SMS), hardware key (YubiKey)	Can be lost, stolen, or (for SMS) intercepted via SIM swap.
Something you are	Fingerprint, face recognition	Can't be "reset" if the biometric data is ever compromised.

How passwords should (and shouldn't) be stored

A password should never be recoverable from the database, not stored in plaintext, and not stored with reversible encryption. It should be hashed: run through a one-way function so that verifying a login means re-hashing the entered password and comparing hashes, never decrypting anything. Two more details make this actually safe: a unique random salt per user (so identical passwords don't produce identical hashes, defeating precomputed rainbow-table attacks) and a deliberately slow algorithm (so that guessing millions of passwords per second becomes computationally expensive).

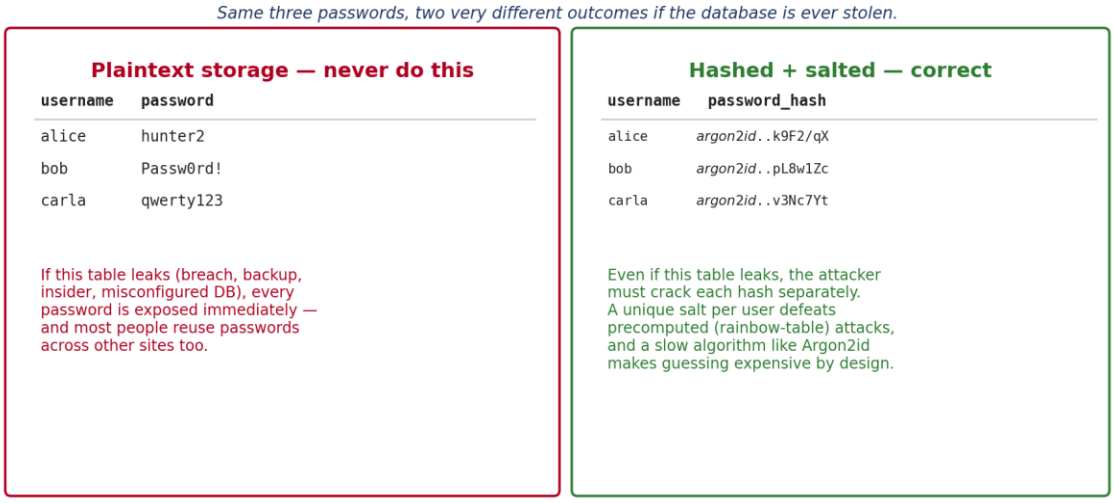


Figure 1 — The same breach, two different outcomes, depending entirely on how passwords were stored.

OWASP's current guidance ranks acceptable algorithms as: Argon2id first choice, scrypt as a fallback, bcrypt for legacy systems only, and PBKDF2 where FIPS-140 compliance is required. Fast general-purpose hashes like plain MD5 or SHA-256 are not acceptable for passwords, they're designed to be fast, which is exactly the wrong property here.

Session management

Once you're authenticated, the server issues a session ID: a long, random, unguessable value and sends it via Set-Cookie (Week 2). From then on, the browser resends that ID with every request, and the server looks it up in its session store to know who's asking. Two operational details matter: sessions should expire after a period of inactivity (an idle timeout), and critically a fresh session ID should always be issued at the moment of login, replacing whatever ID the browser held before. Skipping that second step is precisely what enables the next attack.

Session Fixation

In a session fixation attack, the attacker doesn't need to steal anything, they simply get the victim to authenticate using a session ID the attacker already controls.

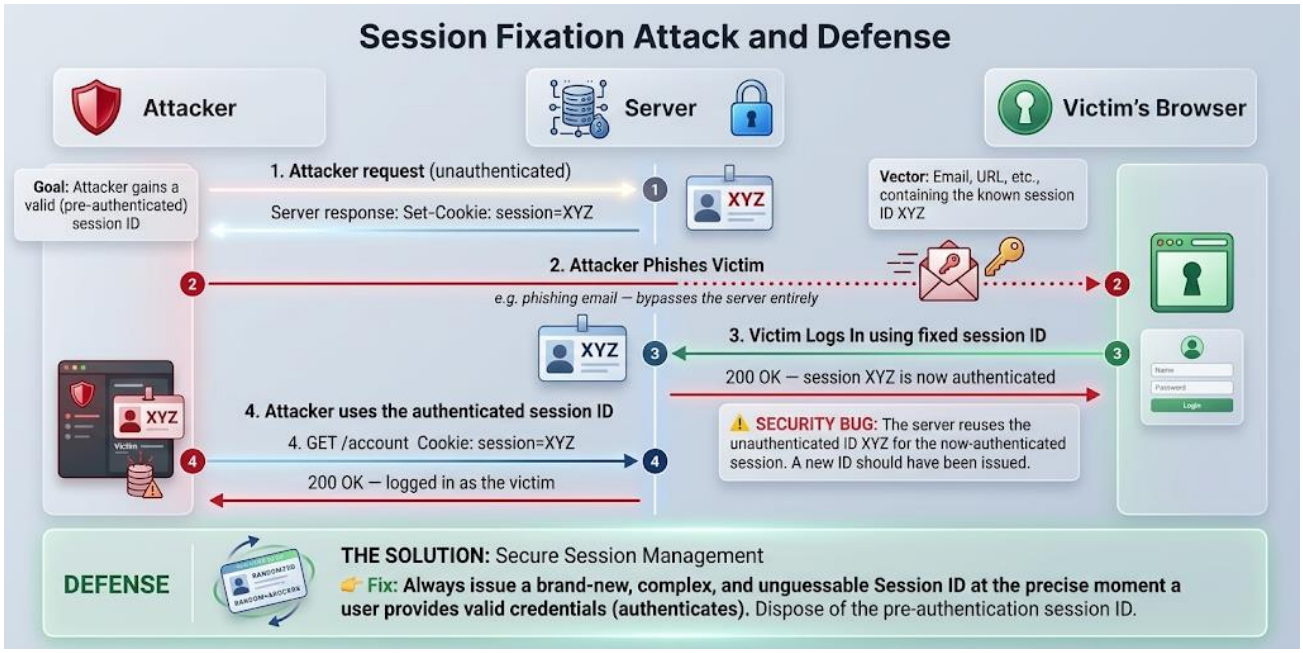


Figure 2 — The attacker never touches the victim's credentials; they only need the server to keep reusing the same session ID after login.

The root cause is the same theme from Week 1 and Week 2: trusting something that should have been regenerated. The fix is one rule, applied consistently: every privilege change logging in, but also elevating to an admin role should issue a brand-new session ID.

Common authentication and session attacks

Attack	How it works	Primary mitigation
Brute Force	Automated tool tries many password guesses against one account.	Account lockout / rate limiting, CAPTCHA, MFA.
Credential Stuffing	Attacker replays username/password pairs leaked from other, unrelated breaches.	MFA, checking new passwords against known-breach lists, per-IP rate limiting.
Session Hijacking	Attacker steals a valid session ID via XSS, network sniffing on plain HTTP, or a leaked log.	HttpOnly + Secure cookies (Week 2), HTTPS everywhere.
Session Fixation	Attacker forces the victim to authenticate while using a session ID the attacker already knows.	Regenerate the session ID the moment a user logs in (see Figure 2).

Modern authentication: tokens, OAuth, and SSO

Classic session cookies require the server to keep a lookup table of every active session. An alternative is token-based authentication, most commonly JSON Web Tokens (JWT): a self-contained token made of three base64url-encoded parts (***header.payload.signature)***. The header names the signing algorithm, the payload carries claims (who the user is, when the token expires), and the signature created with a server-held secret or private key lets the server verify the token wasn't tampered with, without needing to store it anywhere. This trades server-side memory for a new risk: a stolen token is valid until it expires, since there's no session store entry to revoke.

OAuth 2.0 and Single Sign-On (SSO) solve a related but different problem: letting a user authenticate once with a trusted identity provider (Google, Microsoft, your university account) and reuse that identity across multiple applications, instead of creating yet another password to manage and from the application's side, avoiding the responsibility of storing passwords at all.